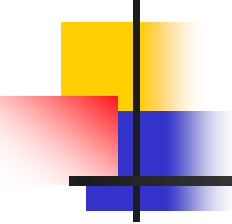




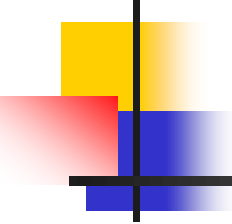
Shellsort

CPS212, Gordon College



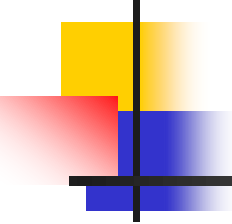
Shellsort

- Invented by Donald Shell in 1959.
- 1st algorithm to break the quadratic time barrier but few years later, a sub quadratic time bound was proven
- Shellsort works by comparing elements that are **distant** rather than adjacent elements in an array.



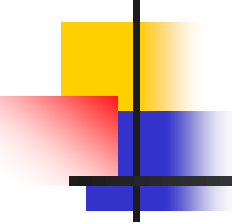
Shellsort

- Shellsort uses a sequence $h_1, h_2, \dots, h_t$ called the ***increment sequence***. Any increment sequence is fine as long as $h_1 = 1$ and some other choices are better than others.



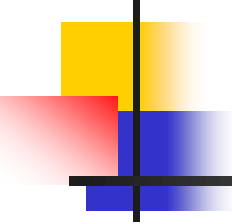
Shellsort

- Shellsort makes multiple passes through a list and sorts a number of equally sized sets using the insertion sort.



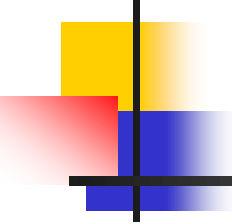
Shellsort

- Shellsort improves on the efficiency of insertion sort by *quickly* shifting values to their destination.



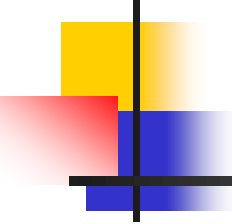
Shellsort

- Shellsort is also known as *diminishing increment sort*.
- The distance between comparisons decreases as the sorting algorithm runs until the last phase in which adjacent elements are compared



Shellsort

- After each phase and some increment h_k , for every i , we have $a[i] \leq a[i + h_k]$ all elements spaced h_k apart are sorted.
- The file is said to be h_k – sorted.



Shellsort Examples

Sort: 18 32 12 5 38 33 16 2

8 Numbers to be sorted, Shell's increment will be $\text{floor}(n/2)$

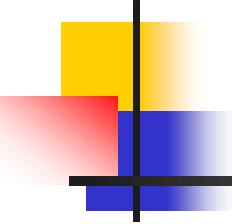
* $\text{floor}(8/2) \rightarrow \text{floor}(4) = 4$

increment 4: 1 2 3 4 (visualize underlining)

18 32 12 5 38 33 16 2

Step 1) Only look at 18 and 38 and sort in order ;
18 and 38 stays at its current position because they are in order.

Step 2) Only look at 32 and 33 and sort in order ;
32 and 33 stays at its current position because they are in order.



Shellsort Examples

Sort: 18 32 12 5 38 33 16 2

8 Numbers to be sorted, Shell's increment will be $\text{floor}(n/2)$

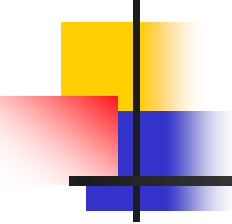
* $\text{floor}(8/2) \rightarrow \text{floor}(4) = 4$

increment 4: 1 2 3 4 (visualize underlining)

18 32 12 5 38 33 16 2

Step 3) Only look at 12 and 16 and sort in order ;
12 and 16 stays at its current position because they are in order.

Step 4) Only look at 5 and 2 and sort in order ;
2 and 5 need to be switched to be in order.



Shellsort Examples (con't)

Sort: 18 32 12 5 38 33 16 2

Resulting numbers after increment 4 pass:

18 32 12 2 38 33 16 5

* $\text{floor}(4/2) \rightarrow \text{floor}(2) = 2$

increment 2: 1 2

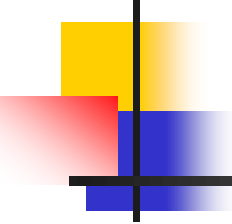
18 32 12 2 38 33 16 5

Step 1) Look at 18, 12, 38, 16 and sort them in their appropriate location:

12 38 16 2 18 33 38 5

Step 2) Look at 32, 2, 33, 5 and sort them in their appropriate location:

12 2 16 5 18 32 38 33



Shellsort Examples (con't)

Sort: 18 32 12 5 38 33 16 2

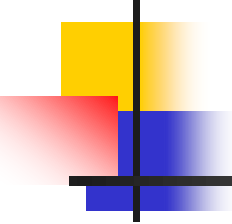
* $\text{floor}(2/2) \rightarrow \text{floor}(1) = 1$

increment 1: 1

12	2	16	5	18	32	38	33
----	---	----	---	----	----	----	----

2	5	12	16	18	32	33	38
---	---	----	----	----	----	----	----

The last increment or phase of Shellsort is basically an Insertion Sort algorithm.



Shell Sort Example

32	95	16	82	24	66	35	19	75	54	40	43	93	68
----	----	----	----	----	----	----	----	----	----	----	----	----	----

Original data

32	95	16	82	24	66	35	19	75	54	40	43	93	68
----	----	----	----	----	----	----	----	----	----	----	----	----	----

Apply 5 sort

32	35	16	68	24	40	43	19	75	54	66	95	93	82
----	----	----	----	----	----	----	----	----	----	----	----	----	----

After 5 sort

32	35	16	68	24	40	43	19	75	54	66	95	93	82
----	----	----	----	----	----	----	----	----	----	----	----	----	----

Apply 3 sort

32	19	16	43	24	40	54	35	75	68	66	95	93	82
----	----	----	----	----	----	----	----	----	----	----	----	----	----

After 3-sort

16	19	24	32	35	40	43	54	66	68	75	82	93	95
----	----	----	----	----	----	----	----	----	----	----	----	----	----

After 1-sort

Shell Sort Example

Insertion sort will take 38 swaps while Shell sort is taking 26 swaps

32	95	16	82	24	66	35	19	75	54	40	43	93	68
----	----	----	----	----	----	----	----	----	----	----	----	----	----

32	95	16	82	24	66	35	19	75	54	40	43	93	68
----	----	----	----	----	----	----	----	----	----	----	----	----	----

32	35	16	68	24	40	43	19	75	54	66	95	93	82
----	----	----	----	----	----	----	----	----	----	----	----	----	----

6 swaps

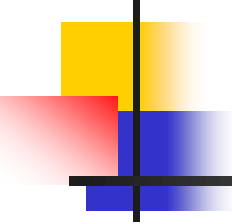
32	35	16	68	24	40	43	19	75	54	66	95	93	82
----	----	----	----	----	----	----	----	----	----	----	----	----	----

32	19	16	43	24	40	54	35	75	68	66	95	93	82
----	----	----	----	----	----	----	----	----	----	----	----	----	----

5 swaps

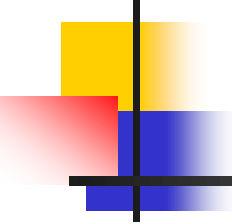
16	19	24	32	35	40	43	54	66	68	75	82	93	95
----	----	----	----	----	----	----	----	----	----	----	----	----	----

15 swaps



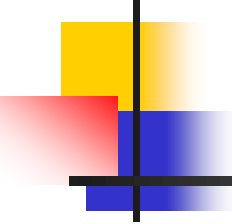
Empirical Analysis of Shellsort (Advantage)

- Advantage of Shellsort is that its only efficient for medium size lists. For bigger lists, the algorithm is not the best choice. Fastest of all $O(N^2)$ sorting algorithms.
- 5 times faster than the bubble sort and a little over twice as fast as the insertion sort, its closest competitor.



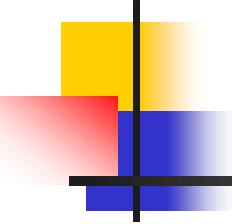
Empirical Analysis of Shellsort (Disadvantage)

- Disadvantage of Shellsort is that it is a complex algorithm and its not nearly as efficient as the merge, heap, and quick sorts.
- The shell sort is still significantly slower than the merge, heap, and quick sorts, but its relatively simple algorithm makes it a good choice for sorting lists of less than 5000 items unless speed important. It's also an excellent choice for repetitive sorting of smaller lists.



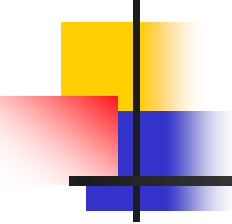
Shellsort Best Case

- Best Case: The best case in the shell sort is when the array is already sorted in the right order. The number of comparisons is less.



Shellsort Worst Case

- The running time of Shellsort depends on the choice of increment sequence.
- The problem with Shell's increments is that pairs of increments are not necessarily relatively prime and smaller increments can have little effect.



Additional Online References

- Animation of Shell Sort
 - <https://www.toptal.com/developers/sorting-algorithms/shell-sort>
- Videos Link
 - <https://www.youtube.com/watch?v=SCBf7aqKQEY>
 - <https://www.youtube.com/watch?v=qzXAVXddcPU>